Roll Number: CB.EN.P2VLD25035, Name: VEDASRI J

Expt No: 3 **Algorithm for computing SCOAP testability measures** **Date:05-03-2026**

Evaluation Table			Faculty Signature
Code:	Output:	Record:	

1. **AIM:** To develop and implement a program that reads a given ISCAS-85 benchmark (.bench) circuit file, analyzes the circuit structure, and computes SCOAP testability measures including combinational controllability (CC0, CC1) and combinational observability (CO) for all nodes to evaluate the testability of the digital circuit.

2. **SOFTWARE/TOOLS USED:** Google Colab (Python)

3. **THEORY:**

3.1 Testability in Digital Circuits

Testability refers to the ease with which internal nodes of a circuit can be controlled and observed during testing. Poor testability makes fault detection difficult.

Two important concepts are:

1. Controllability – Ease of setting a node to a specific value.
2. Observability – Ease of propagating a node value to a primary output.

3.2 SCOAP

It is a structural testability analysis technique used to compute controllability and observability values for each node in a circuit.

SCOAP provides three measures:

i. CC0: Cost of forcing a node to logic 0
ii. CC1: Cost of forcing a node to logic 1
iii. CO: Cost of observing a node at the primary outputs

Higher values indicate lower testability.

3.3 SCOAP Initialization Rules

Primary Inputs: For every primary input, CC0 = 1, CC1 = 1

Primary Outputs: For every primary output, CO = 0

3.4 Controllability Formulas

Gate	CC0	CC1
AND	min(CC0 inputs) + 1	sum(CC1 inputs) + 1
NAND	sum(CC1 inputs) + 1	min(CC0 inputs) + 1
OR	sum(CC0 inputs) + 1	min(CC1 inputs) + 1
NOR	min(CC1 inputs) + 1	sum(CC0 inputs) + 1
NOT	CC1(input) + 1	CC0(input) + 1
BUF	CC0(input) + 1	CC1(input) + 1
XOR	min(CC0A+CC0B , CC1A+CC1B)+1	min(CC0A+CC1B , CC1A+CC0B)+1

3.5 Observability Formulas

Gate	CO(input i)
AND	CO(out) + Σ CC1(other inputs) + 1
NAND	CO(out) + Σ CC1(other inputs) + 1
OR	CO(out) + Σ CC0(other inputs) + 1
NOR	CO(out) + Σ CC0(other inputs) + 1
NOT	CO(out) + 1
BUF	CO(out) + 1
XOR	CO(out) + min(CC0/CC1 other input) + 1

4. ALGORITHM:

1. Read the BENCH circuit file containing the gate-level description.
2. Identify:
   - Primary inputs
   - Primary outputs
   - Internal nodes
3. Construct the fan-in and fan-out graph of the circuit.
4. Initialize:
   - CC0 = 1, CC1 = 1 for primary inputs
   - CO = 0 for primary outputs
5. Perform forward propagation to compute controllability values.
6. Perform backward propagation to compute observability values.
7. Detect stem and branch nodes for fanout structures.
8. Display the computed CC0, CC1, and CO values for each node.
9. Identify the hardest nodes to control and observe.

1. Read .bench file
2. Extract Primary inputs, Primary outputs, Gate definitions
3. Expand fanouts into explicit branch nodes
4. Perform topological sorting
5. Apply test vector
6. Perform true value simulation
7. Initialize primary input fault lists
8. Propagate fault lists gate-by-gate
9. Collect detected faults at primary outputs
10. Compute Total faults, Detected faults, Undetected faults, Fault coverage.

5. CODE:

```
import re
from collections import deque, defaultdict

class TestabilitySCOAP:

    def __init__(self):
        """
        Constructor initializes all the data structures required for
        SCOAP analysis of the circuit.
        """
```

```
    # Dictionary storing all circuit nodes and their properties
    # Each node stores:
    # gate type, fanin nodes, CC0, CC1, CO, and node type
    self.graph = {}

    # List storing all primary inputs
    self.primary_inputs = []

    # List storing all primary outputs
    self.primary_outputs = []

    # Dictionary storing fanout connections
    # Example: node -> list of gates driven by this node
    self.fanout = {}

    # Dictionary storing branch nodes created from stems
    # Example: N3_b1, N3_b2 etc.
    self.branch_nodes = {}

# ---------------------------------------------------
# BENCH FILE READER
# ---------------------------------------------------

def read_bench(self, text):
    """
    Reads and parses the BENCH file describing the circuit.

    BENCH format example:
    INPUT(A)
    OUTPUT(Z)
    N1 = AND(A,B)

    This function extracts:
    - Primary inputs
    - Primary outputs
    - Gates
    - Fanin and fanout relationships
    """

    # Regular expressions to detect different BENCH statements
    input_pattern = re.compile(r'INPUT\((.*?)\)', re.I)
    output_pattern = re.compile(r'OUTPUT\((.*?)\)', re.I)
    gate_pattern = re.compile(r'(.*?)=(.*?)\((.*?)\)', re.I)

    for line in text.splitlines():

        line = line.strip()

        # Ignore empty lines or comments
        if line == "" or line.startswith("#"):
            continue

        # ------------------------
        # PRIMARY INPUT
        # ------------------------
        if input_pattern.match(line):

            node = input_pattern.match(line).group(1).strip()
            self.primary_inputs.append(node)

            # Initialize SCOAP values for primary inputs
            # According to SCOAP rules:
            # CC0 = 1
            # CC1 = 1
```

```
            self.graph[node] = {
                "gate": "INPUT",
                "fanin": [],
                "cc0": 1,
                "cc1": 1,
                "co": float("inf"),   # CO initially unknown
                "type": "PI"
            }

        # ------------------------
        # PRIMARY OUTPUT
        # ------------------------
        elif output_pattern.match(line):

            node = output_pattern.match(line).group(1).strip()
            self.primary_outputs.append(node)

        # ------------------------
        # GATE DEFINITION
        # ------------------------
        elif gate_pattern.match(line):

            out, gtype, ins = gate_pattern.match(line).groups()

            out = out.strip()
            gtype = gtype.strip().upper()

            # Split multiple inputs
            inputs = [i.strip() for i in ins.split(",")]

            # Store node information
            self.graph[out] = {
                "gate": gtype,
                "fanin": inputs,
                "cc0": float("inf"),
                "cc1": float("inf"),
                "co": float("inf"),
                "type": "Internal"
            }

            # Build fanout connections
            for i in inputs:
                self.fanout.setdefault(i, []).append(out)

    # ------------------------
    # MARK PRIMARY OUTPUTS
    # ------------------------
    for node in self.primary_outputs:

        if node in self.graph:

            # If node is both PI and PO
            if self.graph[node]["type"] == "PI":
                self.graph[node]["type"] = "PI/PO"

            else:
                self.graph[node]["type"] = "PO"

    # ------------------------
    # IDENTIFY STEM NODES
    # ------------------------
    # A stem is a node that drives multiple gates (fanout > 1)

    for node, targets in self.fanout.items():

        if len(targets) > 1 and node in self.graph:
```

```
            if self.graph[node]["type"] not in ["PI", "PO", "PI/PO"]:
                self.graph[node]["type"] = "Stem"

# ---------------------------------------------------
# TOPOLOGICAL SORT
# ---------------------------------------------------

def topological_sort(self):
    """
    Generates a topological ordering of nodes in the circuit.

    This ensures nodes are processed only after their inputs
    have been computed.
    """

    indegree = defaultdict(int)

    # Initialize indegree of all nodes
    for node in self.graph:
        indegree[node] = 0

    # Compute indegree from fanin connections
    for node, data in self.graph.items():
        for inp in data["fanin"]:
            indegree[node] += 1

    queue = deque()

    # Nodes with no dependencies start first
    for node in self.graph:
        if indegree[node] == 0:
            queue.append(node)

    topo = []

    # Kahn's algorithm for topological sorting
    while queue:

        n = queue.popleft()
        topo.append(n)

        for out in self.fanout.get(n, []):
            indegree[out] -= 1
            if indegree[out] == 0:
                queue.append(out)

    return topo

# ---------------------------------------------------
# CONTROLLABILITY CALCULATION
# ---------------------------------------------------

def compute_controllability(self):
    """
    Computes controllability values CC0 and CC1 for all nodes.

    Uses SCOAP forward propagation formulas.
    """

    updated = True

    while updated:

        updated = False

        for node, data in self.graph.items():
```

```
            gate = data["gate"]

            # Skip primary inputs
            if gate == "INPUT":
                continue

            inputs = data["fanin"]

            if any(i not in self.graph for i in inputs):
                continue

            in_nodes = [self.graph[i] for i in inputs]

            old_cc0 = data["cc0"]
            old_cc1 = data["cc1"]

            # AND gate formula
            if gate == "AND":

                data["cc0"] = min(i["cc0"] for i in in_nodes) + 1
                data["cc1"] = sum(i["cc1"] for i in in_nodes) + 1

            # NAND gate
            elif gate == "NAND":

                data["cc0"] = sum(i["cc1"] for i in in_nodes) + 1
                data["cc1"] = min(i["cc0"] for i in in_nodes) + 1

            # OR gate
            elif gate == "OR":

                data["cc0"] = sum(i["cc0"] for i in in_nodes) + 1
                data["cc1"] = min(i["cc1"] for i in in_nodes) + 1

            # NOR gate
            elif gate == "NOR":

                data["cc0"] = min(i["cc1"] for i in in_nodes) + 1
                data["cc1"] = sum(i["cc0"] for i in in_nodes) + 1

            # NOT gate
            elif gate == "NOT":

                data["cc0"] = in_nodes[0]["cc1"] + 1
                data["cc1"] = in_nodes[0]["cc0"] + 1

            # BUFFER gate
            elif gate in ["BUF", "BUFF"]:

                data["cc0"] = in_nodes[0]["cc0"] + 1
                data["cc1"] = in_nodes[0]["cc1"] + 1

            # XOR gate
            elif gate == "XOR":

                a, b = in_nodes[0], in_nodes[1]

                data["cc0"] = min(a["cc0"]+b["cc0"], a["cc1"]+b["cc1"]) + 1
                data["cc1"] = min(a["cc0"]+b["cc1"], a["cc1"]+b["cc0"]) + 1

            # Check if values changed
            if old_cc0 != data["cc0"] or old_cc1 != data["cc1"]:
                updated = True
```

```
# ---------------------------------------------------
# OBSERVABILITY CALCULATION
# ---------------------------------------------------

def compute_observability(self):
    """
    Computes observability (CO) for all nodes.

    Uses SCOAP backward propagation formulas.
    """

    # Primary outputs have CO = 0
    for node in self.primary_outputs:
        if node in self.graph:
            self.graph[node]["co"] = 0

    updated = True

    while updated:

        updated = False

        for node, data in self.graph.items():

            out_co = data["co"]

            if out_co == float("inf"):
                continue

            inputs = data["fanin"]
            gate = data["gate"]

            for i in inputs:

                if i not in self.graph:
                    continue

                # Other inputs of the gate
                others = [self.graph[x] for x in inputs if x != i]

                new_co = float("inf")

                # AND / NAND formula
                if gate in ["AND", "NAND"]:
                    new_co = out_co + sum(x["cc1"] for x in others) + 1

                # OR / NOR formula
                elif gate in ["OR", "NOR"]:
                    new_co = out_co + sum(x["cc0"] for x in others) + 1

                # NOT / BUFFER
                elif gate in ["NOT", "BUF", "BUFF"]:
                    new_co = out_co + 1

                # XOR
                elif gate == "XOR":
                    new_co = out_co + min(others[0]["cc0"], others[0]["cc1"]) + 1

                # Update CO if smaller value found
                if new_co < self.graph[i]["co"]:
                    self.graph[i]["co"] = new_co
                    updated = True

                # Create branch nodes for fanout
                if len(self.fanout.get(i, [])) > 1:

                    idx = self.fanout[i].index(node) + 1
```

```
                    bname = f"{i}_b{idx}"

                    self.branch_nodes[bname] = {
                        "cc0": self.graph[i]["cc0"],
                        "cc1": self.graph[i]["cc1"],
                        "co": new_co,
                        "type": "Branch"
                    }

    # Stem observability = minimum branch observability
    for stem in self.fanout:

        if len(self.fanout[stem]) > 1 and stem in self.graph:

            branch_cos = []

            for bname, bdata in self.branch_nodes.items():
                if bname.startswith(stem + "_"):
                    branch_cos.append(bdata["co"])

            if branch_cos:
                self.graph[stem]["co"] = min(branch_cos)

# ----------------------------------------------------
# DISPLAY RESULTS
# ----------------------------------------------------

def display(self):

    topo = self.topological_sort()

    print("\n" + "="*50)
    print("SCOAP TESTABILITY REPORT")
    print("="*50)

    print("\nPrimary Inputs :", ", ".join(self.primary_inputs))
    print("Primary Outputs:", ", ".join(self.primary_outputs))

    print("\nInitialization Assumption:")
    print("For all Primary Inputs → CC0 = 1 and CC1 = 1")

    print("\nObservation Rule:")
    print("For all Primary Outputs → CO = 0")

    # Display each node in topological order
    for node in topo:

        d = self.graph[node]

        cc0 = int(d["cc0"])
        cc1 = int(d["cc1"])
        co = int(d["co"]) if d["co"] != float("inf") else "inf"

        print("\nNode:", node)
        print("Type:", d["type"])
        print("CC0 :", cc0)
        print("CC1 :", cc1)
        print("CO  :", co)
        print("-"*30)

        # Print branch nodes if any
        for b in self.branch_nodes:
            if b.startswith(node + "_"):
                bd = self.branch_nodes[b]

                print("Branch:", b)
                print("CC0 :", int(bd["cc0"]))
```

```
                print("CC1 :", int(bd["cc1"]))
                print("CO  :", int(bd["co"]))
                print("-"*30)

        # ------------------------------------------------
        # FINAL TESTABILITY ANALYSIS
        # ------------------------------------------------

        max_cc0 = -1
        max_cc1 = -1
        max_co = -1

        hardest_cc0 = []
        hardest_cc1 = []
        hardest_co = []

        # Combine nodes and branches
        all_nodes = {**self.graph, **self.branch_nodes}

        for node, d in all_nodes.items():

            cc0 = int(d["cc0"])
            cc1 = int(d["cc1"])
            co = d["co"]

            if cc0 > max_cc0:
                max_cc0 = cc0
                hardest_cc0 = [node]
            elif cc0 == max_cc0:
                hardest_cc0.append(node)

            if cc1 > max_cc1:
                max_cc1 = cc1
                hardest_cc1 = [node]
            elif cc1 == max_cc1:
                hardest_cc1.append(node)

            if co != float("inf"):

                if co > max_co:
                    max_co = co
                    hardest_co = [node]

                elif co == max_co:
                    hardest_co.append(node)

        print("\n" + "="*50)
        print("TESTABILITY ANALYSIS")
        print("="*50)

        print(f"\nHardest nodes to control to 0 (CC0 = {max_cc0}):")
        print(", ".join(hardest_cc0))

        print(f"\nHardest nodes to control to 1 (CC1 = {max_cc1}):")
        print(", ".join(hardest_cc1))

        print(f"\nHardest nodes to observe (CO = {max_co}):")
        print(", ".join(hardest_co))


    # ------------------------------------------------
    # MAIN PROGRAM
    # ------------------------------------------------

    # Ask user to input BENCH file
    file_name = input("Enter BENCH file name: ")
```

```
# Read BENCH file contents
with open(file_name, "r") as f:
    bench_text = f.read()

# Create object of the SCOAP analyzer
scoap = TestabilitySCOAP()

# Parse the BENCH file
scoap.read_bench(bench_text)

# Compute controllability values
scoap.compute_controllability()

# Compute observability values
scoap.compute_observability()

# Display the final SCOAP report
scoap.display()
```

6. SAMPLE INPUT:

c17.bench:

```
# c17
# 5 inputs
# 2 outputs
# 0 inverter
# 6 gates ( 6 NANDs )

INPUT(1)
INPUT(2)
INPUT(3)
INPUT(6)
INPUT(7)

OUTPUT(22)
OUTPUT(23)

10 = NAND(1, 3)
11 = NAND(3, 6)
16 = NAND(2, 11)
19 = NAND(11, 7)
22 = NAND(10, 16)
23 = NAND(16, 19)
```

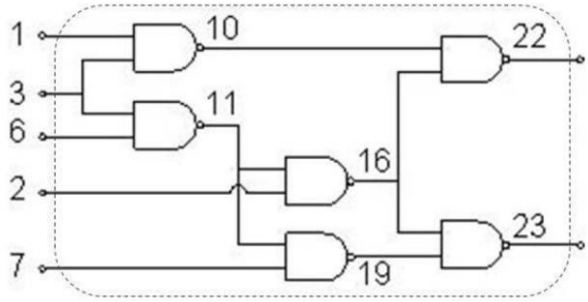

Figure 1: c17 circuit diagram

OUTPUT:

For c17.bench input:

```
Enter BENCH file name: c17.bench

==================================================
SCOAP TESTABILITY REPORT
==================================================

Primary Inputs : 1, 2, 3, 6, 7
Primary Outputs: 22, 23

Initialization Assumption:
For all Primary Inputs → CC0 = 1 and CC1 = 1

Observation Rule:
For all Primary Outputs → CO = 0

Node: 1
Type: PI
CC0 : 1
CC1 : 1
CO  : 5
------------------------------

Node: 2
Type: PI
CC0 : 1
CC1 : 1
CO  : 6
------------------------------

Node: 3
Type: PI
CC0 : 1
CC1 : 1
CO  : 5
------------------------------
Branch: 3_b1
CC0 : 1
CC1 : 1
CO  : 5
------------------------------
Branch: 3_b2
CC0 : 1
CC1 : 1
CO  : 7
------------------------------

Node: 6
Type: PI
CC0 : 1
CC1 : 1
CO  : 7
------------------------------
```

```
Node: 7
Type: PI
CC0 : 1
CC1 : 1
CO  : 6
------------------------------

Node: 10
Type: Internal
CC0 : 3
CC1 : 2
CO  : 3
------------------------------

Node: 11
Type: Stem
CC0 : 3
CC1 : 2
CO  : 5
------------------------------
Branch: 11_b1
CC0 : 3
CC1 : 2
CO  : 5
------------------------------
Branch: 11_b2
CC0 : 3
CC1 : 2
CO  : 5
------------------------------

Node: 16
Type: Stem
CC0 : 4
CC1 : 2
CO  : 3
------------------------------
Branch: 16_b1
CC0 : 4
CC1 : 2
CO  : 3
------------------------------
Branch: 16_b2
CC0 : 4
CC1 : 2
CO  : 3
------------------------------
```

```
Node: 19
Type: Internal
CC0 : 4
CC1 : 2
CO  : 3
------------------------------

Node: 22
Type: PO
CC0 : 5
CC1 : 4
CO  : 0
------------------------------

Node: 23
Type: PO
CC0 : 5
CC1 : 5
CO  : 0
------------------------------

==================================================
TESTABILITY ANALYSIS
==================================================

Hardest nodes to control to 0 (CC0 = 5):
22, 23

Hardest nodes to control to 1 (CC1 = 5):
23

Hardest nodes to observe (CO = 7):
6, 3_b2
```

The **SCOAP testability parameters CC0, CC1, and CO were successfully computed** for all nodes in the given combinational circuit using the implemented Python program. The nodes with the **highest controllability and observability costs were identified**, indicating the **hardest nodes to control and observe during testing**.

7. INFERENCE:

The SCOAP algorithm was implemented to compute controllability (CC0, CC1) and observability (CO) for all nodes in the given digital circuit. The program successfully analyzed the circuit structure and identified nodes that are difficult to control and observe. This analysis helps evaluate the testability of the circuit and locate nodes that may be harder to detect during fault testing.